THE UNLOX ACADEMY

Industry-Graded Minor Project

RedFlag

The Fraud Files

"Build a fraud detection engine using pure SQL. No ML. No Python. No excuses."

INDUSTRY CONNECT

This is what fintech analysts actually do.

You have just been hired as a fraud analyst at PayFast - a fictional Indian payment aggregator handling 200,000 transactions a day. On your desk is one file: six months of raw transaction data. Somewhere in that data, fraudsters are hiding.

Your job is not to build a machine learning model. Your job is to write SQL queries that catch them.

This is the actual job description at Razorpay, Cred, Slice, Jupiter, PhonePe, and every other Indian fintech. Fraud analytics is one of the fastest-growing analyst roles in the country - and it does not require ML. It requires exactly the SQL skills you have been building over the last week.

By the end of these 7 days, you will have caught 12 different fraud patterns. You will produce a single .sql file that can be dropped into any fintech engineering interview and be recognised on sight as competent work.

Time: 7 days

Dataset: 200,000 transactions (single .sql file)

Deliverable: One .sql file with 12 fraud detection queries.

1. Why This Project Exists

For the last week, you learned to filter data, aggregate it, group it, sort it, handle NULLs, and add conditional logic. These are foundational skills. Every analyst has them. Nothing about you is distinctive yet.

RedFlag is the project that changes that.

What separates a candidate who gets hired from one who gets passed over is whether they can apply their SQL to a problem that actually matters. Fraud detection is that problem. It is measurable (either you caught the fraudster or you didn't), industry-relevant (every fintech has a fraud team), and highly technical (real fraud queries combine every skill you learned in Week 3 with joins, subqueries, and window functions from Week 4).

What you will actually build

One SQL file. Twelve sections, one per fraud pattern. Each section contains a query that finds the fraudsters and a comment block explaining what the pattern is and what your query caught.

Every query you write in this project mirrors real queries used at Indian fintech companies today. When you finish, you will have written twelve queries that any fraud team lead at Cred or Slice would recognise immediately as the queries they run daily.

INDUSTRY CONNECT

Why fintech India is hiring fraud analysts aggressively.

In 2024, India processed roughly 130 billion UPI transactions. Fraud on that volume is measured in thousands of crores per year - and it's growing faster than the fraud teams can hire.

Every major Indian fintech (Razorpay, PhonePe, Google Pay, Paytm, BharatPe, Slice, Cred, Jupiter) has an analytics team dedicated to fraud detection. Their day-to-day work is not machine learning. It is SQL. Write queries, catch patterns, produce reports.

A candidate who walks into any of these companies with a working fraud-detection SQL project on GitHub has a huge advantage. Most freshers show up with the same three portfolio projects (calculator, weather app, expense tracker). You will show up with something a fraud team lead can immediately relate to.

2. Your Mission - PayFast Fraud Investigation

You have just joined PayFast, a fictional Indian payment aggregator that processes UPI, card, netbanking, and wallet transactions across 20+ Indian cities. In the past six months, PayFast has processed 200,000 transactions. Buried in that data are twelve distinct fraud patterns being run by suspected fraudsters.

Your job is to identify every fraudster.

The rules of engagement

- **One dataset.** The full 200,000 transaction history is in redflag_transactions.sql - a single file.
- **One table.** All transactions are in a single flat table called transactions. This mirrors real fraud analytics work - production fraud teams often use denormalised tables because joins on live data are too expensive.
- **Pure SQL.** No Python. No Excel. No machine learning libraries. Just SQL queries.
- **12 patterns.** Each one is a real fraud technique used by criminals against real payment systems. Your task is to detect each one with a targeted query.
- **Tiered difficulty.** 5 patterns catchable with Week 3 skills. 5 more need Week 4 joins/subqueries. 2 stretch patterns need window functions from Week 4 Session 4.

Who is the user of your work?

Imagine your query results land on the desk of a fraud operations manager on Monday morning. She will call the flagged accounts, freeze the funds, and escalate the highest-risk cases to law enforcement. Your query IS the tool that triggers those actions. Write queries you would be comfortable defending in a room with your compliance team.

3. Understanding the Dataset

The transactions table has 9 columns. Here is what each one means and what you can expect to see in the data.

Column	Type	What it stores
txn_id	BIGINT (PK)	Unique 8-digit transaction ID, sequential in time
user_id	INT	The account making the transaction. Range: 1-14755.
merchant_id	INT	The merchant receiving the money. Range: 1-800.
amount	DECIMAL(10,2)	Transaction value in Rupees. Range: ₹1 to ₹1,00,000.
txn_time	DATETIME	Timestamp of transaction (2024-01-01 to 2024-06-30)
status	VARCHAR(10)	'SUCCESS' or 'FAILED' — most (~97%) succeed.
payment_mode	VARCHAR(15)	'UPI', 'CARD', 'NETBANKING', or 'WALLET'
city	VARCHAR(30)	20 Indian cities. Most users stick to their home city.
txn_type	VARCHAR(10)	'DEBIT' (money out), 'CREDIT' (money in), 'REFUND'

Dataset scale

- **200,000 transactions** spread over 6 months (Jan-Jun 2024)
- **~14,500 legitimate users** with normal behaviour - city loyalty, reasonable frequency, small-to-medium amounts
- **255+ suspect users** with one or more fraud patterns embedded in their transaction history
- **800 merchants** across 12 categories - most are legitimate, some are colluding with fraudsters

INDUSTRY CONNECT

This is not a toy dataset.

200,000 rows is a full trailer of what production fintech data looks like. In your first month on a fraud analytics team, you will be running queries against millions of rows - sometimes billions.

At this scale, your queries need to be structured well. Poorly written queries (SELECT * without WHERE, cartesian joins, missing indexes) will crash. Well-written queries return in seconds.

PayFast's data has proper indexes on user_id, txn_time, and merchant_id. If your queries take more than 30 seconds, you are probably writing them inefficiently - check for missing WHERE clauses or unnecessary sub-selects.

4. Loading the Dataset - Step by Step

The dataset is a single .sql file called redflag_transactions.sql. It contains one CREATE DATABASE, one CREATE TABLE, and hundreds of INSERT statements. Follow these steps exactly.

Step 1 - Increase MySQL Workbench timeout

Because the file is 18 MB, the default Workbench timeout will kill the import halfway through. Before you do anything else:

1. Open Workbench
2. Edit > Preferences > SQL Editor
3. Set 'DBMS connection read timeout' to 600 (10 minutes)
4. Click OK. Restart Workbench.

Step 2 - Import the SQL file

5. File > Open SQL Script > select redflag_transactions.sql
6. Workbench may warn: 'File is too big to open in the editor. Load anyway?' - click Continue Anyway.
7. Press Ctrl + Shift + Enter to execute the entire script.
8. Wait 30-90 seconds. Do not interrupt. The status bar at the bottom shows progress.

Step 3 - Verify the import worked

Run these three verification queries. They are already at the end of the file.

```
SELECT COUNT(*) FROM redflag.transactions;
-- Expected result: approximately 200,000

SELECT COUNT(DISTINCT user_id) FROM redflag.transactions;
-- Expected result: approximately 14,700

SELECT MIN(txn_time), MAX(txn_time) FROM redflag.transactions;
-- Expected result: 2024-01-01 to 2024-06-30
```

Common problems

- **"Query timed out" -** You did not increase the timeout in Step 1. Do it now, restart Workbench, re-run the script.
- **"Lost connection to MySQL server" -** Same fix. The default read timeout is 30 seconds. Fix it to 600.
- **"Database redflag already exists" -** You ran the script twice. The script's DROP DATABASE at the top handles this - just re-run the whole script.

- **"Cannot open file, size too big" -** Some older Workbench versions have a hard file size limit. Solution: right-click the .sql file, choose 'Run SQL Script' from the command line if available, or use a modern Workbench version (8.x).

5. New MySQL Concepts You Will Need

Week 3 covered most of what you need. There are four additional MySQL functions that this project requires. All are simple - I include them here so you don't have to Google them mid-project.

5.1 DATE() - extract the date part from a datetime

```
SELECT DATE(txn_time) FROM transactions LIMIT 3;
-- Returns: '2024-03-15', '2024-03-16', '2024-03-16'
-- Use in GROUP BY when you want to count transactions per day.
```

5.2 HOUR() - extract the hour (0-23) from a datetime

```
SELECT user_id, HOUR(txn_time) FROM transactions LIMIT 3;
-- Returns hour as integer 0 through 23
-- Useful for time-of-day analysis in Pattern 5.
```

5.3 DATE_FORMAT() - format a datetime in a custom way

```
SELECT DATE_FORMAT(txn_time, '%Y-%m') FROM transactions LIMIT 3;
-- Returns: '2024-03', '2024-03', '2024-04'
-- Useful for GROUP BY month.
```

5.4 TIMESTAMPDIFF() - compute the gap between two datetimes

```
SELECT TIMESTAMPDIFF(MINUTE, '2024-03-15 10:00:00', '2024-03-15 10:45:00');
-- Returns: 45

SELECT TIMESTAMPDIFF(DAY, '2024-01-15', '2024-04-15');
-- Returns: 91
-- Available units: SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, YEAR
```

You do not need any other functions for the mandatory tiers. Tier 3 patterns use window functions which are taught separately.

6. The Three Tiers - Your Progression

The 12 fraud patterns are grouped into three tiers by difficulty. Each tier maps to a different week of the SQL curriculum.

TIER 1

TIER 1 - 5 patterns · attemptable NOW (Week 3 skills)

P1 · Velocity fraud P2 · Round-amount clustering P3 · Card testing

P4 · Failed-then-succeeded pairs P5 · Odd-hour concentration

Skills needed: GROUP BY, HAVING, COUNT, SUM, CASE WHEN, WHERE, IN, BETWEEN — all covered in Week 3.

Marks: 6 per pattern × 5 = 30 marks total.

TIER 2

TIER 2 - 5 patterns · unlocks after Week 4 Sessions 1-3

P6 · Mule accounts P7 · Refund abuse P8 · Merchant collusion

P9 · Just-under-threshold P10 · Dormant-then-active

Skills needed: joins, subqueries, EXISTS, correlated subqueries (Week 4 S1-S3).

Note: several Tier 2 patterns have a SIMPLIFIED version catchable with just Week 3 skills (worth partial marks) and an ADVANCED version requiring Week 4 skills (full marks).

Marks: 8 per pattern × 5 = 40 marks total.

TIER 3

TIER 3 - 2 patterns · unlocks after Week 4 Session 4

P11 · Velocity spike P12 · Geographic impossibility

Skills needed: window functions (LAG, ROW_NUMBER, OVER PARTITION BY).

These are the queries that separate top submissions from average ones. If you get both, you are demonstrating fluency at the level a fraud team lead would expect from a mid-level analyst.

Marks: 10 per pattern × 2 = 20 marks total.

Plus 10 marks for report quality (comments, formatting, cleanliness of the .sql file) = 100 marks total.

Recommended attack order

Do NOT try to solve them in numerical order. Instead:

9. **Days 1-3:** All 5 Tier 1 patterns. Get them working. Get suspect counts. Verify against expected ranges in Section 9.
10. **Days 4-5:** 5 Tier 2 patterns. Start with the simplified versions if Week 4 sessions haven't landed yet; upgrade to advanced versions as you learn joins/subqueries.
11. **Days 6:** 2 Tier 3 patterns after Week 4 Session 4 (Window Functions).
12. **Day 7:** Polish the .sql file. Add comments. Verify suspect counts one final time.

7. The 12 Fraud Patterns

What follows is a briefing on each of the 12 fraud patterns you must detect. For each pattern I give you: what the pattern looks like in real fraud operations, the signature you are looking for in the data, the SQL skills required, and the approximate suspect count you should expect. I do NOT give you the query - writing it is the project.

P1 · Velocity Fraud

TIER 1

Tier 1 · 6 marks

Skills: GROUP BY, DATE(), HAVING, COUNT - all Week 3.

The pattern: A legitimate user makes 3-8 transactions per day on their busiest days. A fraudster running an automated script can make 30+ in a single day. Anyone hitting that count is either a bot, an account takeover, or a merchant running a churning scheme.

The signature: A single user_id with 30 or more distinct transactions on any one calendar date.

Hint: GROUP BY user_id AND DATE(txn_time), then filter groups with HAVING.

Expected suspects: About 45-55 user-days flagged. There are 30 seeded fraudsters plus ~15-25 legitimate power users who occasionally hit the threshold - this false-positive rate is realistic.

P2 · Round-Amount Clustering

TIER 1

Tier 1 · 6 marks

Skills: WHERE, IN, GROUP BY, HAVING - all Week 3.

The pattern: Money launderers prefer round-number amounts (₹100, ₹500, ₹1,000, ₹5,000, ₹10,000). Real e-commerce and food-delivery transactions rarely produce clean round numbers because prices include taxes, delivery fees, and discounts. A user with 15+ exactly-round transactions is showing money-laundering signature.

The signature: A single user_id with 15+ transactions where amount is exactly one of: 100, 200, 500, 1000, 2000, 5000, 10000.

Hint: Use WHERE amount IN (list of round numbers), then aggregate by user.

Expected suspects: Exactly 25 (all seeded).

P3 · Card Testing

TIER 1

Tier 1 · 6 marks

Skills: WHERE, GROUP BY on multiple columns, HAVING - all Week 3.

The pattern: Fraudsters buy dumps of stolen credit card numbers on the dark web. They test which cards are still active by attempting tiny purchases (under ₹10). If the purchase goes through, the card is still valid and the fraudster keeps it for a bigger operation. If it fails, they move to the next card. This is one of the most common frauds detected by real card networks.

The signature: A single user_id with 30+ transactions under ₹10 in a single day.

Hint: Combine a WHERE amount clause with grouping by user AND date. The daily grouping is critical - spreading tiny transactions across weeks is not fraud, concentrating them in one day is.

Expected suspects: Exactly 20 (all seeded).

P4 · Failed-Then-Succeeded

TIER 1

Tier 1 · 6 marks

Skills: WHERE, GROUP BY, HAVING - Week 3. Advanced version uses subqueries.

The pattern: Same card-testing behaviour as P3, but the specific signature this time is many FAILED transactions followed by SUCCESS ones. Fraudsters retry until they find a card/CVV combination that clears. Real users rarely have more than 2-3 failed transactions in an entire year. Users with 20+ failures are running scripts.

The signature (simplified): A single user_id with 20+ transactions where status = 'FAILED'.

The signature (advanced, Week 4): A user_id with 20+ pairs where a FAILED transaction is followed within 2 minutes by a SUCCESS transaction of the same amount.

Hint: The simplified version filters where status = 'FAILED' and counts by user. The advanced version needs a self-join or correlated subquery.

Expected suspects: Exactly 25 (all seeded).

P5 · Odd-Hour Concentration

TIER 1

Tier 1 · 6 marks

Skills: HOUR(), CASE WHEN inside SUM, HAVING with aggregate - Week 3.

The pattern: Real Indian users transact between 8 AM and 11 PM. Automated fraud scripts often run in the 2 AM - 5 AM window (which is business hours in North American timezones - many card-cracking rings operate from Eastern Europe and the Americas). A user with the vast majority of their activity in this window is exhibiting bot signature.

The signature: A user_id where 80% or more of their transactions occur between 2 AM and 5 AM (hours 2, 3, 4), and they have at least 30 total transactions.

Hint: Use SUM(CASE WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1 ELSE 0 END) to count odd-hour transactions. Then compute the ratio in HAVING.

Expected suspects: Exactly 20 (all seeded).

P6 · Mule Accounts

TIER 2

Tier 2 · 8 marks

Skills: subqueries, EXISTS clauses (Week 4). Simplified version uses only GROUP BY.

The pattern: Mule accounts are the human ATMs of the fraud world. A fraudster deposits stolen funds into a mule's account, then quickly withdraws or transfers them elsewhere. The mule keeps a small commission. Behaviour signature: large CREDIT transactions (money coming in via NETBANKING) immediately followed by DEBIT transactions (money going out via UPI) within 30 minutes.

The signature (simplified, Week 3): A user with 8 or more CREDIT transactions. Not perfect, but flags most mules.

The signature (advanced, Week 4): A user with 5+ instances where a CREDIT is followed within 30 minutes by a DEBIT of at least 70% of the credit amount.

Hint: The advanced version uses a correlated subquery: for each CREDIT, check if there EXISTS a matching DEBIT within a time window.

Expected suspects: Exactly 30 (all seeded).

P7 · Refund Abuse

TIER 2

Tier 2 · 8 marks

Skills: CASE WHEN inside aggregate, HAVING with ratios (Week 3, but with more complex logic).

The pattern: Real users have refund rates below 5%. Fraudsters running chargeback schemes or exploiting merchant loopholes have refund rates above 40%. The signature is a user with many transactions where a disproportionate share are refunds.

The signature: A user with 20+ total transactions AND a refund ratio (REFUNDS / TOTAL) greater than 40%.

Hint: SUM(CASE WHEN txn_type = 'REFUND' THEN 1 ELSE 0 END) counts refunds per user. Divide by COUNT(*) for the ratio. Put both conditions in HAVING.

Expected suspects: 24-25 (all seeded).

P8 · Merchant Collusion

TIER 2

Tier 2 · 8 marks

Skills: window functions (ROW_NUMBER) OR subqueries + joins (Week 4).

The pattern: Legitimate merchants have long tails of customers - thousands of users each contributing small amounts to the merchant's total volume. A merchant where 3-4 users generate the majority of volume is either a very niche B2B business (rare on retail platforms) or is colluding with those users to launder money.

The signature: A merchant where the top 5 users by volume account for more than 60% of the merchant's total transaction value.

Hint: Compute the sum of amounts per (merchant, user), rank users within each merchant, sum the top 5 per merchant, compare against merchant total. This is a multi-step CTE query - great practice for Week 4 Session 4.

Expected suspects: Exactly 15 merchants (merchant IDs 1-15 are the seeded colluding merchants).

P9 · Just-Under-Threshold (Structuring)

TIER 2

Tier 2 · 8 marks

Skills: WHERE with exact match, GROUP BY, HAVING - Week 3.

The pattern: Indian banking regulations require enhanced KYC checks on transactions of ₹10,000 or above. Fraudsters running structuring / smurfing schemes deliberately keep transactions at exactly ₹9,999 to avoid these checks. This is one of the most classic anti-money-laundering patterns and is illegal even without any other fraud.

The signature: A user with 10 or more transactions at exactly ₹9,999.00.

Hint: Straightforward - WHERE amount = 9999.00, GROUP BY user, HAVING count >= 10.

Expected suspects: Exactly 20 (all seeded).

P10 · Dormant-Then-Active

TIER 2

Tier 2 · 8 marks

Skills: window function LAG(), OR self-join with subquery (Week 4).

The pattern: An account that was completely inactive for 90+ days and then suddenly bursts with 15+ transactions in a short window is the signature of account takeover. The fraudster has gained access to a dormant account (via a phishing attack, credential leak, or SIM swap) and is monetising it before the real owner notices.

The signature: A user who has a gap of 90+ days between two consecutive transactions, followed by 15+ transactions after the gap.

Hint: Use LAG(txn_time) OVER (PARTITION BY user_id ORDER BY txn_time) to compute the gap between each user's consecutive transactions. Filter for gaps >= 90 days. Then count post-gap transactions per user.

Expected suspects: 25-27 (25 seeded + occasional noise).

P11 · Velocity Spike

TIER 3

Tier 3 · 10 marks

Skills: window functions, CTEs (Week 4 Session 4).

The pattern: A user's transaction rate suddenly spikes to many multiples of their historical average. This is the ML-free equivalent of anomaly detection - even without training a model, you can identify accounts whose behaviour changed abruptly. Almost always indicates account takeover.

The signature: A user whose peak monthly transaction count is at least 5x their average monthly transaction count (and peak is at least 20 transactions).

Hint: Build a CTE that computes monthly transaction counts per user. Then a second CTE computes each user's average and peak. Filter users whose peak/average ratio exceeds 5.

Expected suspects: 35-45. 20 seeded users MUST appear in results. Additional 15-25 clean users with low baselines and one busy month naturally trigger this signature - mirroring real production data.

P12 · Geographic Impossibility

TIER 3

Tier 3 · 10 marks

Skills: window function LAG(), TIMESTAMPDIFF (Week 4 Session 4).

The pattern: The same user transacts in two different Indian cities within 60 minutes. Physically impossible unless the account is being used simultaneously by two different people. Almost always indicates account takeover or stolen-card usage across a syndicate.

The signature: A user_id where at least one pair of consecutive transactions occurs in different cities within 60 minutes of each other.

Hint: Use LAG(city) and LAG(txn_time) OVER (PARTITION BY user_id ORDER BY txn_time) to attach each transaction's previous city and time. Filter where cities differ AND TIMESTAMPDIFF(MINUTE, prev_time, txn_time) <= 60.

Expected suspects: Exactly 15 (all seeded).

8. What You Will Submit

Your submission is a single .sql file called RedFlag_<YourName>.sql. Inside are twelve sections, one per fraud pattern, each with a properly commented query.

Required structure - copy this template

```
-- ======================================================================
-- RedFlag — Fraud Detection Submission
-- Student: <Your Name>  |  Batch: DA-DS-1
-- ======================================================================

USE redflag;

-- ======================================================================
-- PATTERN 1 · VELOCITY FRAUD
-- What I'm looking for: users with 30+ transactions in a single day
-- Expected suspects: ~50
-- ======================================================================

SELECT user_id, DATE(txn_time) AS attack_date, COUNT(*) AS daily_txn_count
FROM transactions
GROUP BY user_id, DATE(txn_time)
HAVING COUNT(*) >= 30
ORDER BY daily_txn_count DESC;

-- My findings: 52 suspect user-days flagged.
-- Top 3 fraudsters by transaction count: user 14523 (45 txns on 2024-04-12),
-- user 14508 (44 txns on 2024-02-28), user 14515 (43 txns on 2024-05-19).

-- ======================================================================
-- PATTERN 2 · ROUND-AMOUNT CLUSTERING
-- (etc — repeat for all 12 patterns)
-- ======================================================================
```

Requirements per pattern

- **Comment header -** one paragraph describing what the pattern is and what you're looking for.
- **The query itself -** readable formatting. Keywords in UPPERCASE. Clauses on new lines. Sensible column aliases.
- **Findings comment -** one paragraph after the query with your suspect count and a couple of specific examples (user IDs, dates).

What to leave out

- Do NOT include the CREATE TABLE or INSERT statements from the dataset file. Your file should assume the redflag database already exists.
- Do NOT include exploratory queries you wrote to understand the data. Only the final detection queries.
- Do NOT include queries that don't run. Every query in your file must execute without errors. Test each one before submitting.

9. Grading - And Why You Can't ChatGPT Your Way Through This

Every pattern in this dataset has a KNOWN correct suspect count. Not approximate - verified. When we grade your submission, we run your query and check the returned count against the expected range.

Grading rubric - how marks are awarded per pattern

Result	Marks awarded (of pattern's max)
Suspect count within expected range AND query is well-structured	100% (full marks)
Suspect count within expected range BUT query has minor issues (missing alias, no ORDER BY, wrong-order clauses)	80%
Query attempts the right pattern but count is 20-50% off due to wrong threshold or missing filter	50%
Query addresses the pattern but has major logic error (e.g., no GROUP BY where required)	30%
Query does not execute, or addresses wrong pattern, or blank	0%

Total marks

- Tier 1 (5 patterns × 6 marks): 30 marks
- Tier 2 (5 patterns × 8 marks): 40 marks
- Tier 3 (2 patterns × 10 marks): 20 marks
- Code quality + comments: 10 marks
- Total: 100 marks. Pass: 60. Distinction: 85+.

Anti-plagiarism

- Two submissions with more than 70% similarity in query structure = both get zero.
- Suspect counts wildly outside expected ranges without explanation = zero for that pattern.
- AI-assisted queries: permitted for syntax help only. If the whole submission looks AI-generated (uniform commenting style, no exploration comments, perfect but uniform structure), we will discount the marks accordingly.

10. The 7-Day Build Plan

Day 1 - Setup + Data Exploration (2 hours)

- Import redflag_transactions.sql. Verify 200,000 rows loaded.
- Run 10-15 exploratory queries to understand the data. Suggestions: distribution of statuses, distribution of payment modes, distribution of amounts (min/max/avg), transactions per city, count of unique users, count of unique merchants.
- Read this brief front-to-back. Get comfortable with the tier structure.

Day 2 - Tier 1: Patterns 1-3 (2 hours)

- P1: Velocity fraud. Focus: GROUP BY (user, date), HAVING >= 30.
- P2: Round-amount clustering. Focus: WHERE amount IN(list), GROUP BY user, HAVING >= 15.
- P3: Card testing. Focus: WHERE amount < 10, GROUP BY (user, date), HAVING >= 30.
- Verify suspect counts against the expected ranges in Section 7. If off, debug.

Day 3 - Tier 1: Patterns 4-5 (2 hours)

- P4: Failed-then-succeeded. Focus: WHERE status = 'FAILED', GROUP BY user.
- P5: Odd-hour concentration. Focus: HOUR(txn_time), CASE WHEN inside SUM, ratio in HAVING.
- By end of day 3, ALL 5 Tier 1 patterns should be complete. That's 30 marks banked before Week 4 even begins.

Day 4 - Tier 2: Patterns 6-8 (3 hours)

If Week 4 Session 1 (Joins) has been delivered, you can attempt the advanced versions of these. Otherwise use the simplified Week 3 versions for partial credit.

- P6: Mule accounts. Simplified: WHERE txn_type = 'CREDIT', GROUP BY user, HAVING >= 8. Advanced: correlated subquery.
- P7: Refund abuse. CASE WHEN inside SUM, HAVING with ratio > 40%.
- P8: Merchant collusion. Requires multi-CTE query with ROW_NUMBER - attempt after Week 4 S4.

Day 5 - Tier 2: Patterns 9-10 (2 hours)

- P9: Just-under-threshold. Straightforward: WHERE amount = 9999.00, GROUP BY user, HAVING >= 10.
- P10: Dormant-then-active. Uses LAG() - attempt after Week 4 Session 4.

Day 6 - Tier 3: Patterns 11-12 (2 hours)

These require window functions from Week 4 Session 4.

- P11: Velocity spike. Multi-CTE with monthly aggregates and peak/avg comparison.
- P12: Geographic impossibility. LAG() to get previous city and time, filter for city change within 60 min.

Day 7 - Polish + Verify + Submit (2 hours)

- Clean up your .sql file. Ensure every query has proper comments and formatting.
- Re-run all 12 queries. Note actual suspect counts in your comments.
- Delete exploratory queries.
- Screenshot your favourite query's output - for LinkedIn (Section 11).
- Submit via the Google Form Girish shares in Slack.

11. The Showcase - LinkedIn Strategy

Every project you complete at Unlox is an audition for your first job. RedFlag is no different - arguably it's the most job-relevant project you'll build in the entire course.

The post

Once you finish, post to LinkedIn using this template. Screenshot your best query's output (probably P8 or P11 - they look the most impressive).

```
I spent a week building a fraud detection engine using pure SQL.

The dataset: 200,000 transactions from a simulated Indian
payment aggregator (Razorpay-style).

The mission: catch 12 different fraud patterns hidden in the data.

Patterns I detected:
- Velocity fraud (30+ txns per user per day)
- Card testing (30+ sub-Rs 10 txns in one day)
- Money laundering via round-amount clustering
- Mule accounts (fast in/out transfers)
- Structuring at just-under-KYC-threshold Rs 9999
- Account takeover via velocity spike + geographic impossibility
- ...and 6 more

The tools: MySQL. That's it. No Python. No ML.
No fintech APIs. Just SQL queries with GROUP BY, HAVING, CASE WHEN,
correlated subqueries, and window functions.

This is what fraud analysts at Cred / Slice / Razorpay / Jupiter
do every day. Now I can do it too.

GitHub repo with all 12 queries: [link]

#SQL #DataAnalytics #Fintech #FraudDetection #UnloxAcademy
```

GitHub repo structure

Push your project as a public GitHub repo with:

- **README.md -** one-paragraph project description, screenshot of one query result, list of patterns detected, tech stack
- **RedFlag_YourName.sql -** your submission file
- **screenshots/ -** 3-4 screenshots of best query outputs

- **DO NOT push redflag_transactions.sql -** the 18 MB dataset file. Link to it via Drive or leave a note that recruiters can request it.

INDUSTRY CONNECT

Why fraud detection projects land recruiter DMs.

Fraud detection is one of the highest-priority hiring areas at every Indian fintech. Recruiters actively search LinkedIn for candidates with fraud-detection experience - even simulated experience like this project.

A recruiter at Razorpay who sees 'built fraud detection engine using SQL' on your profile has a much easier time defending your candidacy internally than one who sees 'built expense tracker app'. The former is directly aligned with a live open role. The latter is not.

Historically, students who post RedFlag-style projects on LinkedIn have received 3-5 recruiter DMs within 2 weeks. This is not the norm for portfolio projects. It is specific to how fintech-adjacent the work is.

12. Frequently Asked Questions

Q: The Workbench import keeps timing out. What now?

Increase the read timeout to 600 seconds as described in Section 4, restart Workbench, and try again. If that still fails, try importing via command line: mysql -u root -p < redflag_transactions.sql (this bypasses Workbench entirely).

Q: I'm not getting the expected suspect counts. Is my query wrong?

Probably yes, but not necessarily. Some patterns have expected RANGES (e.g., P1 expects 45-55, not exactly 30). If your count is within the range, you're fine. If it's wildly outside (e.g., 200 or 5), your query has a bug — usually a missing GROUP BY column, missing WHERE filter, or a wrong threshold.

Q: Can I use Python to help analyse the data?

No. The submission is a .sql file. Every query must be SQL. You may use Python for personal exploration if it helps you think, but nothing from Python may appear in your submission.

Q: I don't have Week 4 sessions yet. Can I still submit early?

Yes - you can submit Tier 1 only for 30 marks. But we strongly recommend waiting for the full 7 days and completing all tiers. Tier 3 alone is worth 20 marks.

Q: What if a suspect user matches multiple patterns?

That's realistic and expected. Real fraudsters exhibit multiple signatures. Your queries should still detect each pattern independently.

Q: Can I use ChatGPT for hints?

For syntax help ("how does DATE_FORMAT work in MySQL"), yes. For entire queries, no — and we can tell from your suspect counts if you tried. See Section 9 on grading.

Q: My query works but is slow (30+ seconds). Is that OK?

For most patterns, queries should complete in 3-10 seconds. If a query takes 30+ seconds, you probably have an inefficient subquery or a missing WHERE filter. Efficiency is not directly graded but slow queries usually mean bugs.

Q: Do I need to catch ALL suspects, or just some?

Your query should return the full list of suspects matching the pattern's signature. If your query returns 3 users when it should return 25, marks are deducted proportionally. Section 9 has the grading rubric.

Q: Should I submit even if I only got Tier 1 working?

Yes. 30 marks is a valid submission. Doing nothing is 0. Half-finishing is 30. Attempt what you can.

Q: What's the pass mark?

60/100. Which means either all 5 Tier 1 patterns + 4 Tier 2 patterns (30 + 32 = 62), OR all 5 Tier 1 + 3 Tier 2 + 1 Tier 3 (30 + 24 + 10 = 64), OR similar combinations. Tier 1 alone is not enough - you need to attempt at least some Tier 2 work.

13. A Note Before You Start

This is the project you have been given in this course. It is also the most rewarding.

The dataset is 20 times larger than anything you have queried before. The queries are more complex than anything you have written before. And the stakes - even simulated - are higher. Every suspect you catch is a fraudster you take off the streets. Every one you miss is money laundered through a mule account.

You will get stuck. Some patterns will take you longer than others. Pattern 8 (merchant collusion) has broken about a third of every previous cohort until they realise it needs a multi-step CTE. Pattern 12 (geographic impossibility) will feel impossible until Week 4 Session 4 teaches you LAG(). This is intentional - these are the queries that make you feel like a real analyst.

When you finish, you will have something no other student in your batch will have: a working fraud detection engine, in pure SQL, on realistic Indian fintech data. Post it on LinkedIn. Push it to GitHub. Talk about it in interviews. This project is a legitimate audition for your first fintech analytics role.

Good hunting.

— The Unlox Academy Team